Spring 2026

Object-Oriented Programming

Lecture 5: OOP II

Prof. Jaehyung Seo

(seojae777@konkuk.ac.kr)

Language Intelligence & Representation Lab.

- **Midterm (Wed, April 22) 09:00 – 10:50**

- Most questions will be based on the course materials and my lecture notes

- Question formats will primarily include True/False (+1 for correct, -1 for incorrect, 0 for blank), Multiple Choice, and Free-Response (or Short Answer)

- Answers may be written in English/Korean
- Scope: All materials covered before the midterm

- In person written exam (closed book, no devices)

Overview of Lecture 5

Recognizing Objects
- Determining when to use built-in types vs. custom classes.

Properties
- getter/setter problem
- Blurring the line between data (attributes) and behavior (methods).

Manager Objects
- Creating classes that orchestrate other objects.

The DRY Principle
- Utilizing inheritance and composition to eliminate "copy-pasta" code.

Built-in Data Structures vs. Custom Classes

Built-in Data Structures	Custom Classes
Used when you only need to store data.	Used when you have both data and behavior.
No user-defined methods or domain-specific behavior	Encapsulates specific domain logic.
Simple, fast, and built into Python.	Highly structured and extensible.

Data and Functions Separated (Non-OOP)

Code Snippet:

```
# A polygon as a list of tuples + standalone functions
import math
# Data: A polygon modeled as a list of point tuples
square = [(1,1), (1,2), (2,2), (2,1)]

# Behavior: Standalone functions
def distance(p1, p2):
    return math.sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)

def perimeter(polygon):
    points = polygon + [polygon[0]]
    return sum(distance(points[i], points[i+1])
               for i in range(len(polygon)))
```

Transitioning to Object-Oriented Design

Before (Procedural)
- Disconnected variables and global functions passing data around.
- Data format assumptions with no structural guarantee.

After (OOP)
- A Point class handling distance calculation.
- A Polygon class aggregating points and calculating its own perimeter.
- Behavior is bound directly to the data it operates on.

The Object-Oriented Polygon

Code Snippet:

```
import math
class Point:
    def __init__(self, x: float, y: float):
        self.x, self.y = x, y
    def distance(self, p2: 'Point') -> float:
        return math.sqrt((self.x-p2.x)**2 + (self.y-p2.y)**2)

class Polygon:
    def __init__(self):
        self.vertices = []
    def add_point(self, point: Point):
        self.vertices.append(point)
    def perimeter(self) -> float:
        pts = self.vertices + [self.vertices[0]]
        return sum(pts[i].distance(pts[i+1])
                   for i in range(len(self.vertices)))
```

The Getter/Setter Anti-Pattern

The Problem with Java-Style Access

- In Java, attributes are hidden behind getName() and setName() methods (JavaBeans convention).

- This enforces **encapsulation**: hiding internal representation and enabling validation, serialization, and framework compatibility.

- In Python, this is considered **unpythonic** and clutters the code.

- Python has a much more elegant solution: **Properties**

We are all consenting adults here... right?

The Unpythonic Way

Code Snippet:

```
# Explicit getter/setter methods (like Java)
class Color:
    def __init__(self, rgb_value, name):
        self._rgb_value = rgb_value
        self._name = name
    def set_name(self, name):
        if not name: raise ValueError("Invalid")
        self._name = name
    def get_name(self):
        return self._name

c = Color("#ff0000", "bright red")
c.set_name("red")
print(c.get_name())  # Clunky syntax!
```

Python Properties

Definition

- The property() built-in uses the **descriptor protocol** to intercept attribute access and redirect it to getter/setter methods.

How It Works

- You type c.name = "red"
- Python invokes the property's __set__ method, which calls _set_name("red").

Benefit

- Clean dot-notation syntax + validation logic running behind the scenes
- Uniform Access Principle

Using the property() Function

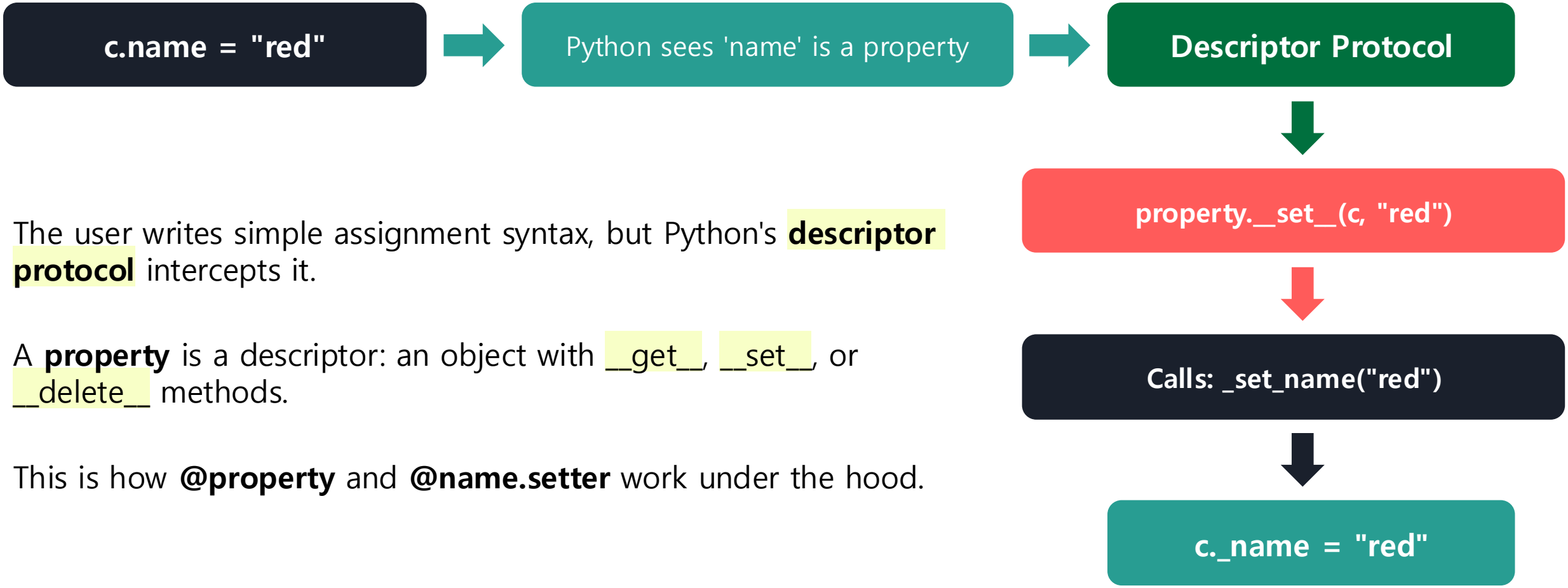
Code Snippet:

```
class Color:
    def __init__(self, rgb_value, name):
        self._rgb_value = rgb_value
        self._name = name
    def _set_name(self, name):
        if not name: raise ValueError("Invalid")
        self._name = name
    def _get_name(self):
        return self._name

    # The Magic: Mapping methods to an attribute
    name = property(_get_name, _set_name)

c = Color("#ff0000", "bright red")
c.name = "red"      # Secretly calls _set_name
print(c.name)       # Secretly calls _get_name
```

How property() Works Internally



The user writes simple assignment syntax, but Python's **descriptor protocol** intercepts it.

A **property** is a descriptor: an object with __get__, __set__, or __delete__ methods.

This is how **@property** and **@name.setter** work under the hood.

The @property Decorator

Using @property Decorator Syntax

- Decorators provide a cleaner, more readable syntax for creating properties.
- Decorate methods with @property, @name.setter, and @name.deleter
- Groups the property logic directly with the method definitions.

Example: Getter + Setter

```
  @property
  def color(self):                          # getter
      return self._color
  @color.setter
  def color(self, value):                   # setter
      self._color = value
```

Read-Only Property

```
  @property
  def area(self):                           # no setter = read-only
      return 3.14 * self.radius ** 2
  c.area = 50  # AttributeError!
```

The Modern Property Syntax

Code Snippet:

```
class Silly:
    def __init__(self):
        self._silly = ""
    @property
    def silly(self):
        print("Getting silly")
        return self._silly
    @silly.setter
    def silly(self, value):
        print(f"Setting silly to {value}")
        self._silly = value

s = Silly()
s.silly = "funny" # Setting silly to funny
print(s.silly)    # Getting silly -> funny
```

When to Use Properties

Decision Tree

- Does the attribute hold simple data?
  → Use standard attributes.

- Do you need to validate input upon assignment?
  → Use a @name.setter property.

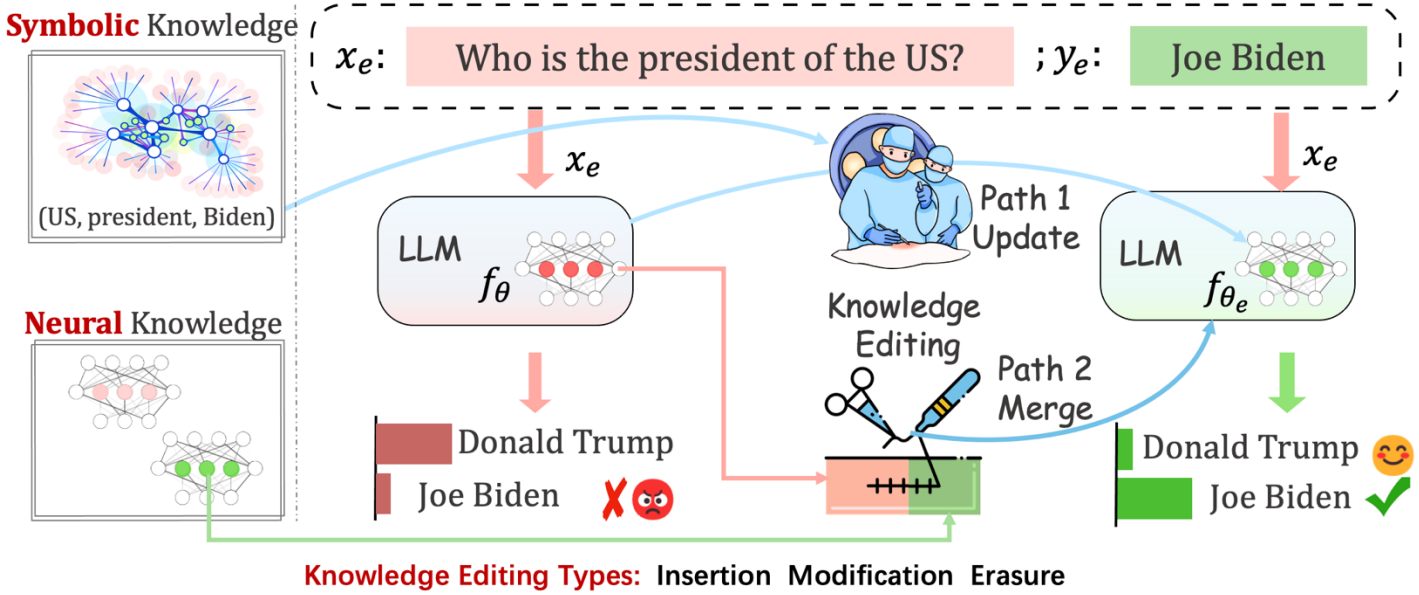
- Does the attribute need to be calculated on-the-fly?
  → Use a @property (getter only).

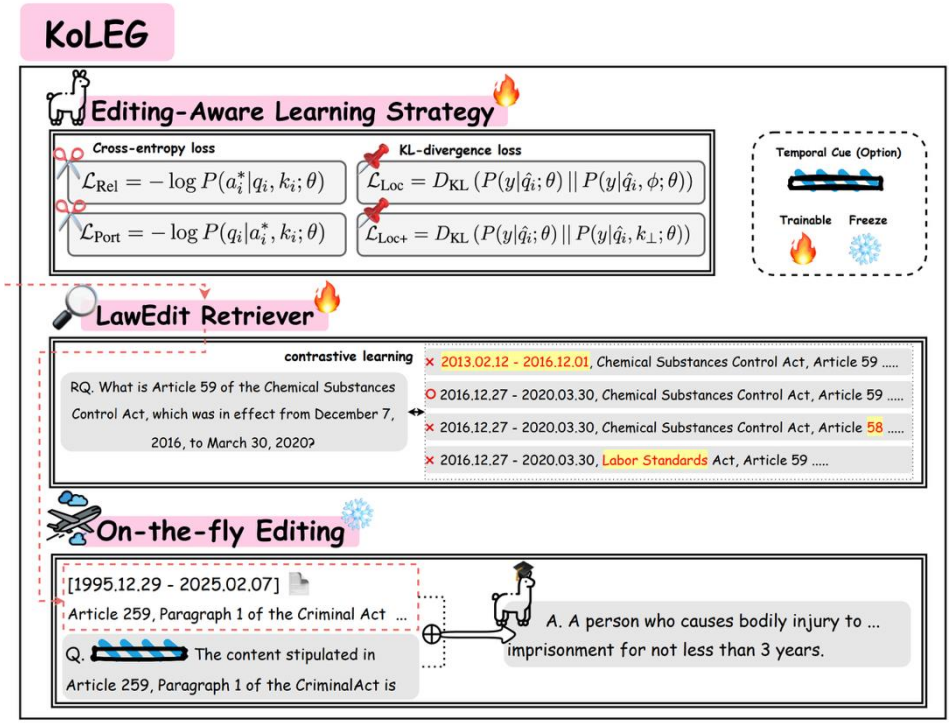
- Does calculating it take a long time?
  → Cache the result inside the property.

On-the-fly?

KoLEG: On-the-Fly Korean Legal Knowledge Editing with Continuous Retrieval

Jaehyung Seo[1], Dahyun Jung[1], Jaewook Lee[1], Yongchan Chun[1], Dongjun Kim[1]
Hwijung Ryu[2], Donghoon Shin[2] and Heuiseok Lim[1†]
[1]Department of Computer Science and Engineering, Korea University
[2]KT Corporation
{seojae777,dhaabb55,jaewook133,cyc9805,junkim100,limhseok}@korea.ac.kr
{hwijung.ryu,dong-hoon.shin}@kt.com





Dynamically Computed Properties

Code Snippet:

```
# A read-only property that calculates an average on the fly
class AverageList(list):
    @property
    def average(self):
        return sum(self)/len(self) if self else 0

grades = AverageList([80, 90, 100])
print(grades.average)  # 90.0
grades.append(100)
print(grades.average)  # 92.5 (Recalculated!)
```

The Role of Manager Objects

Think of an Orchestra Conductor

- Manager objects do **not** perform low-level tasks themselves.

- They tie everything together: managing state, delegating messages, and organizing the workflow of other objects.

- Hold references to worker objects and delegate tasks at the right time.

- Encapsulate the overall workflow of the application.

Orchestration – AI Agent

SYMPHONY: Synergistic Multi-agent Planning with Heterogeneous Language Model Assembly

Wei Zhu, Zhiwen Tang*, Kun Yue

School of Information Science and Engineering, Yunnan University, Kunming, China
Yunnan Key Laboratory of Intelligent Systems and Computing, Kunming, China
zhuwei@stu.ynu.edu.cn, {zhiwen.tang, kyue}@ynu.edu.cn

NEURAL INFORMATION PROCESSING SYSTEMS

雲南大學 信息学院 SCHOOL of INFORMATION SCIENCE & ENGINEERING

1. Motivation & Problem

Existing approaches for LLM-based planning (e.g., in MCTS) predominantly rely on a single agent.

- This paradigm queries the same model repeatedly, assuming stochasticity alone provides sufficient exploration.
- In practice, this leads to low reasoning diversity, as outputs often reflect the same "dominant reasoning pattern."
- This results in narrow search trajectories, suboptimal planning, and susceptibility to local optima.
- **Need:** A multi-agent, heterogeneous reasoning framework to improve exploration and robustness.

2. SYMPHONY Framework

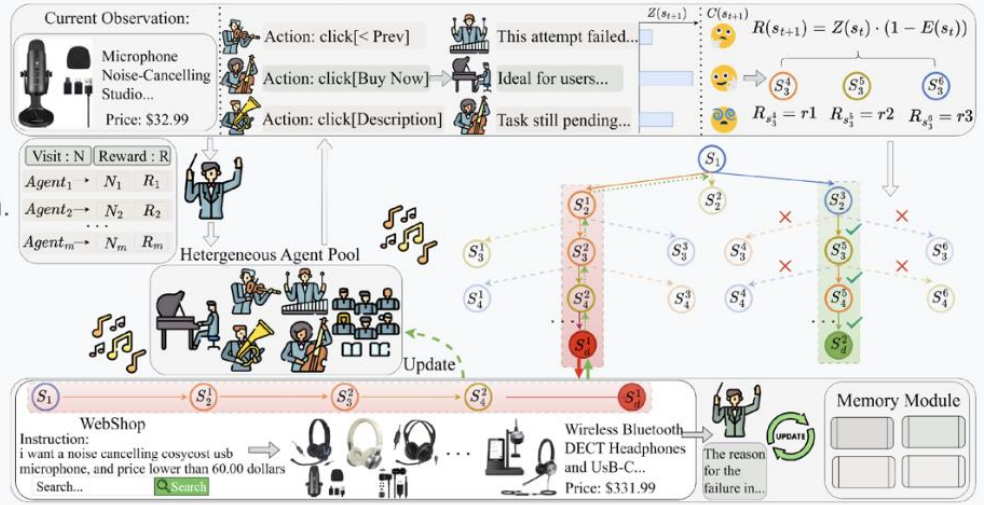
Key Idea: Integrate multiple heterogeneous LLMs into a unified MCTS-based planner.

1 **Heterogeneous Agent Pool:** The core engine providing diverse reasoning styles (e.g., GPT-4, DeepSeek, Qwen).

2 **UCB-based Agent Scheduling:** Dynamically selects agents, balancing exploration and exploitation.

3 **Pool-wise Memory Sharing:** Enables collective learning. Failed trials yield shared reflections to avoid repeated errors.

4 **Entropy-Modulated Confidence Scoring:** Calibrates value estimates by agent confidence for stable search.



Building a Manager (Setup)

Code Snippet:

```
# Skeleton of a ZipProcessor manager class
import zipfile
from pathlib import Path

class ZipProcessor:
    def __init__(self, archive: Path):
        self.archive = archive
        self.temp_dir = Path(f"unzipped_{archive.stem}")
    def process_zip(self):
        self.unzip_files()
        self.process_files()
        self.zip_files()
    def process_files(self):
        pass  # Override in subclasses
```

Extending the Manager

Steps to Utilize the ZipProcessor

Step 1: Inherit from the ZipProcessor manager class.

Step 2: Initialize with specific parameters (search string, replace string).

Step 3: Override process_files to execute specific business logic.

Key Benefit

- Subclass only implements the specific action; parent handles file I/O.

The Subclass Implementation

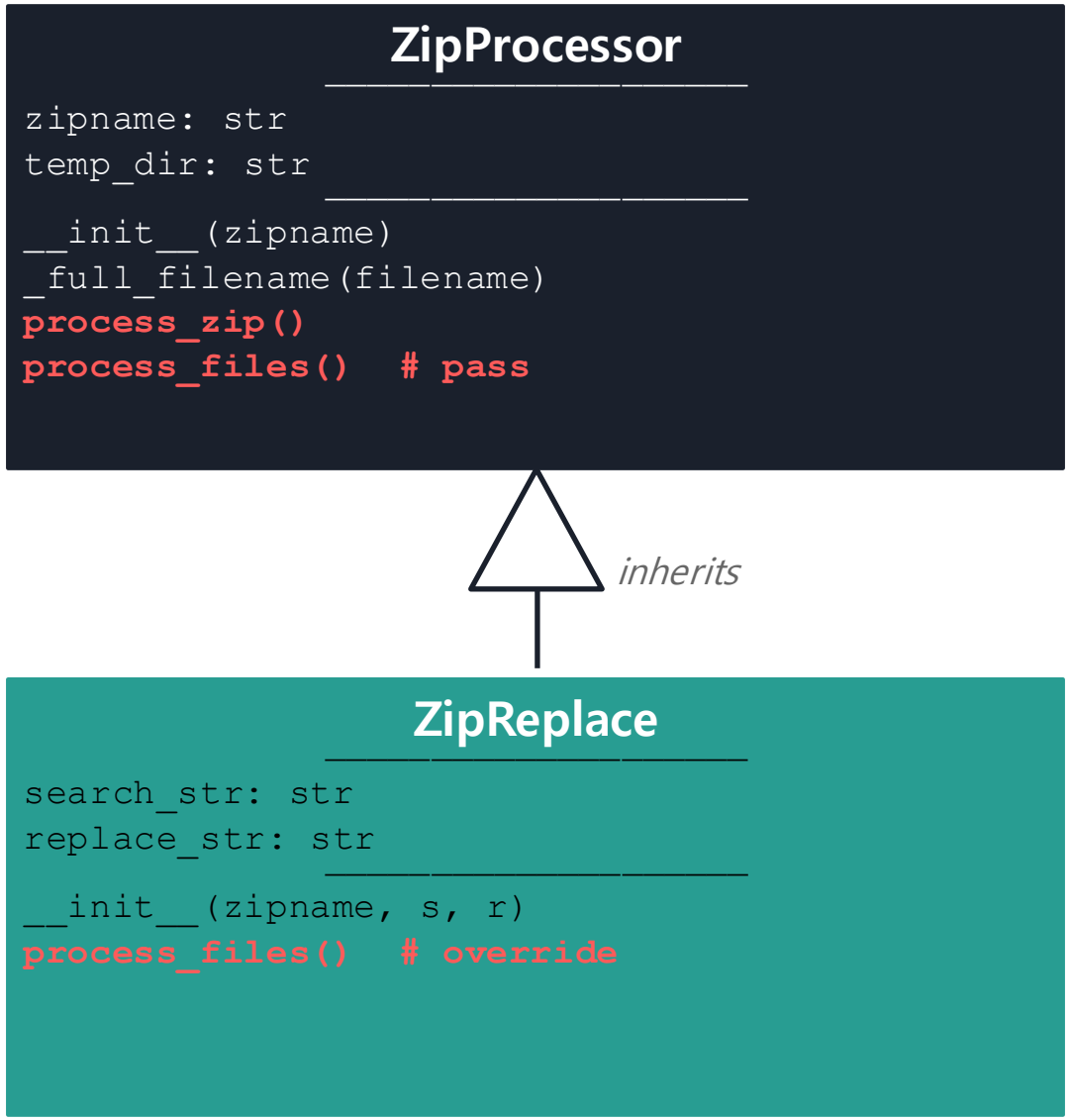
Code Snippet:

```
class ZipReplace(ZipProcessor):
    def __init__(self, archive, search, replace):
        super().__init__(archive)
        self.search = search
        self.replace = replace

    def process_files(self):
        for f in self.temp_dir.rglob("*"):
            if f.is_file():
                txt = f.read_text()
                txt = txt.replace(self.search, self.replace)
                f.write_text(txt)

# ZipReplace(Path("data.zip"),"old","new").process_zip()
```

ZipProcessor Hierarchy



Template Method Pattern

process_zip() defines the **skeleton**. Subclasses override **process_files()** to fill in the details.

The DRY Principle

Don't Repeat Yourself

Recall: In Lecture 4, we applied DRY by moving from_dict() into the base Sample class.

The Pitfall (Copy-Pasta)

- Copying zip extraction logic to make a new class (e.g., ZipImageResize).
- If a bug is found, you must fix it in **multiple places**.

The Solution (Manager Superclass)

- Abstract the common workflow into a manager superclass.
- Use OOP inheritance to enforce single source of truth.

In Practice – A Document Editor

Aggregation Relationship

Document (The Manager)
- Contains characters (A list of Character objects)
- Contains Cursor (An object managing position logic)

Key Design Principle
- Instead of one monolithic class, compose it of smaller, focused objects.
- Cursor handles movement; Document manages the overall interaction.

Implementing the Document Editor

Code Snippet:

```
class Cursor:
    def __init__(self, document):
        self.document = document
        self.position = 0
    def forward(self):
        self.position += 1

class Document:
    def __init__(self):
        self.characters = []
        self.cursor = Cursor(self)
    def insert(self, character):
        self.characters.insert(
            self.cursor.position, character)
        self.cursor.forward()
```

Summary

Design is about deciding when to apply OOP tools.

- Do not force classes on simple data.
- Use @property to elegantly wrap state changes.
- Build Manager objects to coordinate complex workflows.
- Keep code clean, maintainable, and DRY.